



Corso di Laurea Magistrale in Ingegneria Elettronica

Progettazione di Sistemi Integrati per l'IoT (6 CFU)

Rilevamento Microclimi in una Biblioteca



Docenti

Prof. Sergio Saponara
Prof. Daniele Rossi
Ing. Ettore Nocetti

Studenti

Alessio Manuelli
Mattia Bernini
Samuele Bartorelli

Indice

1. Introduzione.....	2
1.1 Scelta delle soglie.....	2
2. Programmazione.....	3
2.1 Componenti sulla scheda.....	3
2.1.1 Sensore di temperatura e umidità HTS221.....	4
2.1.2 Sensore di luminosità BH1750FVI.....	8
2.2 Server.....	9
2.3 Client ESP32.....	12
2.4 Client smartphone.....	14
2.4.1 Interfaccia applicazione.....	15
3. Test e risultati.....	17
3.1 RSSI in funzione della distanza e della Ptx.....	17
3.2 Potenza dissipata.....	18
3.2.1 Impatto del data rate sul consumo di potenza.....	19
4. Antenna PIFA.....	21
4.1 Simulazione con HFSS.....	21
5. Altre soluzioni sul mercato.....	26
6. Conclusioni.....	27
7. Bibliografia.....	28

1. Introduzione

La conservazione ottimale del patrimonio librario è una priorità per biblioteche, archivi e musei, in particolare quando si tratta di opere antiche o di valore storico. Questi ambienti richiedono un monitoraggio costante di parametri ambientali come temperatura, umidità e luminosità per garantire condizioni ideali di conservazione.

Esistono sul mercato diverse soluzioni commerciali per il controllo ambientale, come il sistema Testo Saveris 2, che sfruttano data logger Wi-Fi e architetture cloud. Tuttavia, molte di queste soluzioni sono caratterizzate da costi elevati e infrastrutture chiuse. Una panoramica più dettagliata è fornita nel capitolo 5.

In questo contesto, il nostro progetto propone un sistema IoT a basso consumo energetico, basato su microcontrollori ESP32 e tecnologia Bluetooth Low Energy (BLE), pensato per ambienti in cui la scalabilità e il controllo diretto sono prioritari. Il progetto ha quindi l'obiettivo di sviluppare un sistema per il monitoraggio dei microclimi all'interno di una biblioteca. In particolare, si prevede la rilevazione di parametri ambientali come temperatura, umidità e luminosità mediante una scheda ESP32 configurata come server. I dati raccolti verranno letti da un client, che li confronterà con soglie predefinite per assicurarsi che restino entro limiti ottimali, essenziali per la corretta conservazione dei libri.

1.1 Scelta delle soglie

I libri antichi (fino al XIX secolo) e i libri moderni (dal XX secolo in poi) sono realizzati con materiali differenti, che reagiscono in modo diverso a fattori ambientali come umidità, temperatura e luce. Di conseguenza, ciascuna categoria necessita di specifiche condizioni di conservazione per preservarne l'integrità nel tempo.

I libri antichi sono generalmente prodotti con carta fatta a mano, spesso composta da fibre di lino o cotone, che risultano più resistenti. Gli inchiostri utilizzati sono a base di ferro, legati con materiali naturali, mentre le rilegature sono realizzate in cuoio o pergamena. Al contrario, i libri moderni sono prodotti con carta industriale, spesso acida e derivata dal legno, utilizzano inchiostri chimici e presentano copertine in materiali plastificati o cartonati.

Di seguito viene proposta una tabella riassuntiva delle condizioni ambientali ideali per la corretta conservazione di libri antichi e moderni, distinguendo i parametri di temperatura, umidità e illuminazione.

Fattore	Libri Antichi	Libri Moderni
Temperatura	16 - 18 °C	18 - 21 °C
Umidità	45 - 55 %	40 - 60 %
Luminosità	< 50 lux	fino a 150 lux

Fig. 1: Condizioni di corretta conservazione

2. Programmazione

Sono stati sviluppati tre codici: uno per il server e due per due diverse soluzioni client. Il server è basato su una scheda ESP32 che, una volta stabilita la connessione BLE (Bluetooth Low Energy), legge su richiesta i dati da tre sensori.

Per quanto riguarda il client, sono state proposte due soluzioni:

1. La prima prevede l'uso di un'altra scheda ESP32, che si connette al server e ogni 10 secondi mostra i dati rilevati sul display.
2. La seconda utilizza un'applicazione per smartphone, sviluppata da noi, che consente di leggere i dati dal server premendo un pulsante, dopo aver stabilito la connessione.

In entrambe le soluzioni, è stato implementato un sistema di avviso per segnalare all'utente eventuali superamenti delle soglie di sicurezza, tramite messaggi visualizzati sul display.

2.1 Componenti sulla scheda

Di seguito è mostrata un'immagine della scheda con evidenziati i principali componenti impiegati per il nostro progetto:

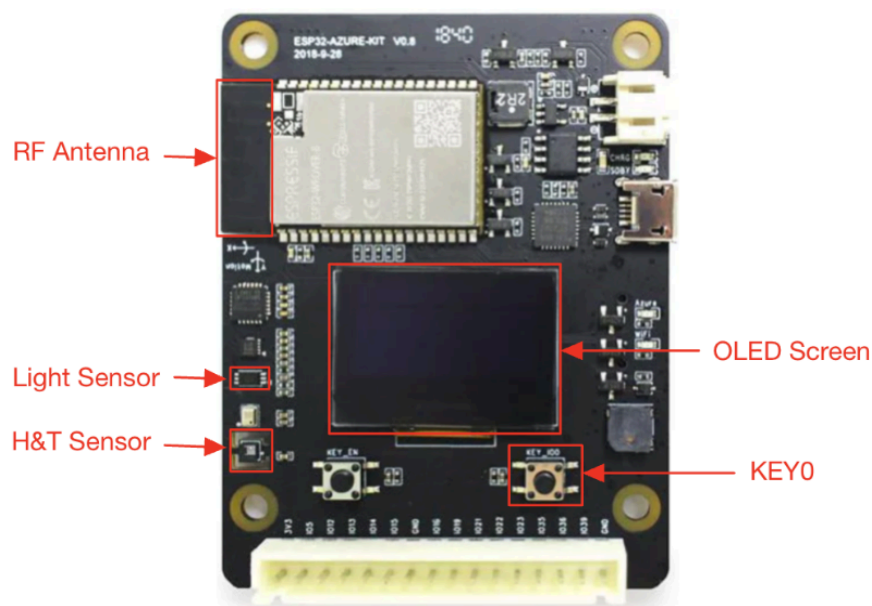


Fig. 2: Scheda ESP32

Il ruolo dei vari componenti è:

- **RF Antenna:** componente responsabile della comunicazione wireless.
- **Light Sensor (BH1750FVI):** sensore di luminosità, che misura l'intensità della luce ambientale in lux.

- **H&T Sensor (HTS221):** sensore combinato di temperatura e umidità.
- **OLED Screen:** display OLED, usato per visualizzare i valori letti dai sensori.
- **KEY0:** pulsante fisico presente sul lato server; la sua pressione avvia la procedura di connessione.

2.1.1 Sensore di temperatura e umidità HTS221

Il sensore HTS221 utilizza l'interfaccia I2C per comunicare e consente di misurare sia la temperatura che l'umidità relativa. Le sue accuratèzze tipiche sono di $\pm 0.5^{\circ}\text{C}$ per la temperatura e $\pm 3.5\%$ per l'umidità.

Di seguito l'indirizzo I2C per comunicare con il sensore.

```
#define HTS221_SENSOR_ADDR 0x5F
```

Fig. 3: Indirizzo I2C del sensore di temperatura e umidità

Per gestire la comunicazione con il sensore, sono state implementate due funzioni: una per scrivere e una per leggere i registri. Durante la lettura, è stato necessario impostare il bit 7 dell'indirizzo del registro a 1 (operazione OR con 0x80) per abilitare la modalità di auto-incremento, utile quando si devono leggere più byte consecutivi.

Di seguito riportiamo la configurazione dei vari registri di configurazione del sensore:

Registro HTS221_AV_CONF (0x10) al quale abbiamo assegnato il valore 0x1B, permette di effettuare la media su 32 campioni per l'umidità e su 16 per la temperatura. Questo garantisce una lettura più stabile.

7	6	5	4	3	2	1	0
Reserved	AVGT2	AVGT1	AVGT0	AVGH2	AVGH1	AVGH0	

To configure humidity/temperature average.

Fig. 4: Registro HTS221_AV_CONF

Registro HTS221_CTRL_REG1 (0x20) al quale abbiamo assegnato il valore 0x85, permette al sensore di acquisire i dati 1 volta al secondo, ed evitare letture incoerenti tra LSB e MSB.

7	6	5	4	3	2	1	0
PD	Reserved				BDU	ODR1	ODR0

[7]	PD: power-down control (0: power-down mode; 1: active mode)
[6:3]	Reserved
[2]	BDU: block data update (0: continuous update; 1: output registers not updated until MSB and LSB reading)
[1:0]	ODR1, ODR0: output data rate selection (see table 17)

Fig. 5: Registro HTS221_CTRL_REG1

La configurazione dei vari bit è la seguente:

- **Bit 7 (PD):** 1 → Sensore attivo
- **Bit 6–3:** 0000 (non usati)
- **Bit 2 (BDU):** 1 → Block Data Update attivo. Blocca l'aggiornamento dei dati fino a quando non si leggono entrambi i byte (MSB e LSB).
- **Bit 1–0 (ODR):** 01 → Output Data Rate di 1 Hz

Registro HTS221_CTRL_REG2 (0x21) al quale abbiamo assegnato il valore 0x80, viene utilizzato per eseguire un soft reset del sensore al momento dell'inizializzazione. Questo garantisce che vengano ricaricate le impostazioni di default e i parametri di calibrazione interni.

7	6	5	4	3	2	1	0
BOOT	Reserved					Heater	ONE_SHOT
[7]		BOOT: Reboot memory content (0: normal mode; 1: reboot memory content)					
[6:2]		Reserved					
[1]		Heater (0: heater disable; 1: heater enable)					
[0]		One-shot enable (0: waiting for start of conversion; 1: start for a new dataset)					

Fig. 6: Registro HTS221_CTRL_REG2

La configurazione dei vari bit è la seguente:

- **Bit 7 (BOOT):** 1 → Riavvio del sensore
- **Bit 6–0:** 0000000 → Tutti a 0 (nessuna funzione aggiuntiva attiva)

Registro HTS221_CTRL_REG3 (0x22) al quale abbiamo assegnato il valore 0x00, disabilita tutte le interruzioni. Il sensore opera quindi in modalità polling, ovvero i dati vengono letti manualmente tramite il microcontrollore.

Misura della temperatura

Per misurare la temperatura con il sensore **HTS221**, è stata implementata la funzione `hts221_get_temperature()`. Questa si occupa di leggere i dati grezzi dai registri del sensore, recuperarne i parametri di calibrazione e calcolare la temperatura reale attraverso un'interpolazione lineare. Di seguito sono descritti i passaggi eseguiti dalla funzione e i registri utilizzati.

1. Lettura del dato grezzo di temperatura (T_OUT)

Il primo passo consiste nel leggere i **2 byte** del valore grezzo attuale di temperatura. I registri coinvolti sono:

- **HTS221_TEMP_OUT_L (0x2A)** che contiene il byte meno significativo (LSB)

- **HTS221_TEMP_OUT_H (0x2B)** che contiene il byte più significativo (MSB)

Questi due byte vengono combinati in un unico valore intero **T_out** a 16 bit con segno, rappresentante la misura raw del sensore.

2. Lettura dei valori grezzi di calibrazione (T0_OUT e T1_OUT)

Il sensore fornisce due punti di riferimento noti (T0 e T1) per la calibrazione. Si leggono quindi:

- **HTS221_T0_OUT_L (0x3C)** e **HTS221_T0_OUT_H (0x3D)** per la ricostruzione del valore **T0_out**
- **HTS221_T1_OUT_L (0x3E)** e **HTS221_T1_OUT_H (0x3F)** per la ricostruzione del valore **T1_out**

Anche questi sono valori grezzi a 16 bit con segno, rappresentanti il valore di output che il sensore produce a due temperature note.

3. Lettura dei valori di calibrazione in °C (T0 e T1)

Il sensore non fornisce direttamente le temperature T0 e T1 in gradi Celsius, ma dei valori scalati che vanno ricostruiti:

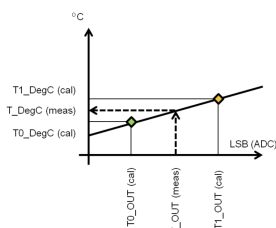
- **HTS221_T0_degC_x8 (0x32)**: 8 bit LSB di T0, moltiplicato per 8
- **HTS221_T1_degC_x8 (0x33)**: 8 bit LSB di T1, moltiplicato per 8
- **HTS221_T1_T0_MSB (0x35)**: contiene i 2 bit MSB di T0 (bit 1:0) e di T1 (bit 3:2)

Unendo LSB e MSB, si ricostruiscono i valori completi a **10 bit** e si dividono per 8.0 per ottenere le temperature in gradi Celsius:

- $T0_degC = T0_x8 / 8.0$
- $T1_degC = T1_x8 / 8.0$

4. Calcolo della temperatura reale

Infine, viene applicata una **interpolazione lineare** tra i due punti di calibrazione per calcolare la temperatura reale come riportato sul datasheet:



$$\text{temperature} = (T1_{degC} - T0_{degC}) \cdot \frac{T_{out} - T0_{out}}{T1_{out} - T0_{out}} + T0_{degC}$$

Fig. 7: Interpolazione lineare per il calcolo della temperatura reale

Misura dell'umidità

Per il sensore di umidità i passaggi sono molto simili. Per la misura è stata realizzata la funzione `hts221_get_humidity()`. Questa legge il dato grezzo dal sensore, utilizza i parametri di calibrazione memorizzati nei registri interni e calcola il valore reale di umidità tramite un'interpolazione lineare.

1. Lettura del dato grezzo di umidità (H_OUT)

Il primo passo consiste nel leggere il valore grezzo attuale dell'umidità. I registri coinvolti sono:

- **HTS221_HUM_OUT_L (0x28)** che contiene il byte meno significativo (LSB)
- **HTS221_HUM_OUT_H (0x29)** che contiene il byte più significativo (MSB)

I due byte vengono uniti per ottenere il valore **H_out**, un intero a 16 bit con segno che rappresenta la misura grezza dell'umidità.

2. Lettura dei valori grezzi di calibrazione (H0_OUT e H1_OUT)

Il sensore HTS221 include due punti di calibrazione interni per l'umidità, H0 e H1. I relativi valori grezzi si trovano nei seguenti registri:

- **HTS221_H0_T0_OUT_L (0x36)** e **HTS221_H0_T0_OUT_H (0x37)** per la ricostruzione del valore **H0_out**
- **HTS221_H1_T0_OUT_L (0x3A)** e **HTS221_H1_T0_OUT_H (0x3B)** per la ricostruzione del valore **H1_out**

Anche questi sono valori a 16 bit con segno, e rappresentano l'output del sensore ai due punti di calibrazione noti.

3. Lettura dei valori di calibrazione in %RH (H0 e H1)

A differenza della temperatura, per l'umidità i valori di calibrazione in %RH sono già memorizzati in forma semplificata: **moltiplicati per 2** e in **8 bit**.

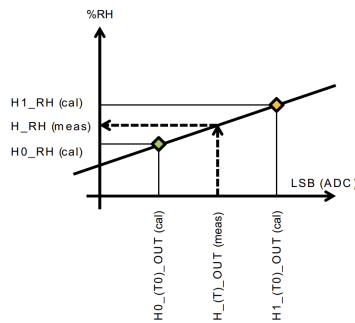
I registri utilizzati sono:

- **HTS221_H0_RH_x2 (0x30)** che contiene il valore di H0 in formato %RH * 2
- **HTS221_H1_RH_x2 (0x31)** che contiene il valore di H1 in formato %RH * 2

I valori vengono convertiti nei corrispondenti **H0_RH** e **H1_RH** in percentuale dividendo per 2.0.

4. Calcolo dell'umidità reale

Una volta ottenuti tutti i parametri, viene applicata la **formula di calibrazione**:



$$\text{humidity} = (H1_{RH} - H0_{RH}) \cdot \frac{H_{out} - H0_{out}}{H1_{out} - H0_{out}} + H0_{RH}$$

Fig. 8: Interpolazione lineare per il calcolo dell'umidità reale

Questa **interpolazione lineare** converte il valore grezzo H_out in un valore reale in percentuale di umidità relativa.

Per rispettare le specifiche del sensore (come indicato dal datasheet), il valore ottenuto viene **limitato tra 0.0% e 100.0%** per compensare eventuali deviazioni dovute a errori di misura o calibrazione.

2.1.2 Sensore di luminosità BH1750FVI

Il sensore BH1750FVI utilizza l'interfaccia I2C per comunicare e consente di misurare la luminosità ambientale con un'elevata precisione, fino a 0.5 lux in modalità High Resolution.

Di seguito è riportato l'indirizzo I2C utilizzato per comunicare con il sensore.

```
#define BH1750FVI_SENSOR_ADDR 0x23
```

Fig. 9: Indirizzo I2C del sensore di luminosità

Per gestire la comunicazione con il BH1750FVI, si utilizzano dei comandi specifici. Questi comandi vengono inviati tramite la scrittura I2C e non è necessario impostare bit particolari per l'auto-incremento.

Di seguito riportiamo i comandi di configurazione del sensore utilizzati:

BH1750FVI_POWER_ON (0x01): Comando per accendere il sensore. Serve per uscire dalla modalità Power Down e preparare il sensore a ricevere comandi di misura.

Power On	0000_0001	Waiting for measurement command.
----------	-----------	----------------------------------

Fig. 10: Comando di accensione del sensore

BH1750FVI_CONTINUOS_MESAURE_HR (0x10): Comando per avviare la misurazione continua in modalità High Resolution.

Continuously H-Resolution Mode	0001_0000	Start measurement at 1lx resolution. Measurement Time is typically 120ms.
--------------------------------	-----------	--

Fig. 11: Comando di avvio della misurazione

Per misurare la luminosità con il sensore BH1750FVI è stata creata la funzione `bh1750fvi_get_luminosity()`. Questa funzione esegue i seguenti passaggi:

1. **Lettura dei dati grezzi:** la funzione legge 2 byte dal sensore tramite la funzione `bh1750fvi_read()`, che esegue la comunicazione I2C con il BH1750FVI per ottenere il valore raw della luminosità.
2. **Ricostruzione del valore a 16 bit:** i due byte letti vengono combinati in un unico valore a 16 bit (`Lx_aux`) con uno shift a sinistra del byte più significativo e un'operazione OR con il byte meno significativo.
3. **Conversione in Lux:** il valore raw viene convertito in Lux dividendo per 1.2, come indicato dal datasheet del sensore. Questa conversione normalizza il dato grezzo in un valore reale.
4. **Saturazione del valore:** viene applicata una saturazione per evitare valori superiori a 600 Lux. Se il valore calcolato supera 600 Lux, viene forzato a 600. Questo evita di avere valori non rappresentabili su 16 bit. Nel nostro caso il valore 600 va più che bene in quanto la massima soglia alla quale i libri possono essere conservati è 150.

2.2 Server

All'avvio della scheda, viene visualizzata la seguente schermata iniziale:



Fig. 12: Schermata iniziale (Server)

Per avviare la ricerca di connessione con il client è necessario premere il pulsante **KEY0** situato sulla scheda. Fino a quando la connessione non viene stabilita, sul display verrà mostrato il seguente messaggio:



Fig. 13: Schermata di attesa di connessione (Server)

Una volta stabilita la connessione, il server, su richiesta del client, esegue le misurazioni delle grandezze richieste, visualizzando i valori letti sul display come mostrato di seguito:

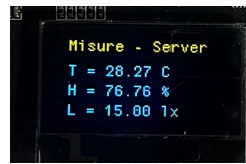


Fig. 14: Schermata di presentazione a connessione stabilita (Server)

Per trasmettere i dati, la scheda utilizza il protocollo BLE, una tecnologia wireless a basso consumo progettata per applicazioni che richiedono comunicazioni frequenti ma a breve raggio. In particolare, il sistema si basa sui ruoli **GAP** e **GATT**.

L'inizializzazione dello stack Bluetooth e la configurazione delle funzioni di callback del driver si trovano nel main:

```
void app_main(void)
{
    ESP_ERROR_CHECK(nvs_flash_init()); // Inizializza la partizione NVS (Non-Volatile Storage) nella memoria flash.
    ESP_ERROR_CHECK(i2c_master_init()); // Inizializza il controller I2C come master per comunicare con i sensori
    wait_for_button_press(); // Attende la pressione di un pulsante prima di procedere con l'inizializzazione del BLE

    // Configura il sensore HTS221 (temperatura e umidità) scrivendo i valori appropriati nei suoi registri
    ESP_ERROR_CHECK(HTS221_register_write_byte(HTS221_AV_CONF, HTS221_AV_CONF_VAL));
    ESP_ERROR_CHECK(HTS221_register_write_byte(HTS221_CTRL_REG1, HTS221_CTRL_REG1_VAL));
    ESP_ERROR_CHECK(HTS221_register_write_byte(HTS221_CTRL_REG2, HTS221_CTRL_REG2_VAL));
    ESP_ERROR_CHECK(HTS221_register_write_byte(HTS221_CTRL_REG3, HTS221_CTRL_REG3_VAL));

    // Configura il sensore BH1750FVI (luminosità) per l'accensione e la misurazione continua in alta risoluzione
    ESP_ERROR_CHECK(bh1750fvi_write_byte(BH1750FVI_POWER_ON));
    ESP_ERROR_CHECK(bh1750fvi_write_byte(BH1750FVI_CONTINUOUS_MEASURE_HR));

    esp_bt_controller_config_t bt_cfg = BT_CONTROLLER_INIT_CONFIG_DEFAULT();
    ESP_ERROR_CHECK(esp_bt_controller_init(&bt_cfg)); // Inizializzazione del controller Bluetooth (HCI, Link Layer e Physical Layer)
    ESP_ERROR_CHECK(esp_bt_controller_enable(ESP_BT_MODE_BLE)); // Abilita il controller Bluetooth in modalità BLE (Bluetooth Low Energy)
    ESP_ERROR_CHECK(esp_bluedroid_init()); // Inizializzazione dello stack software Bluedroid (crea il thread gestore del Bluetooth).
    ESP_ERROR_CHECK(esp_bluedroid_enable()); // Abilita lo stack Bluedroid.

    // Registra le funzioni di callback per gli eventi GAP e GATTs. Queste funzioni verranno chiamate dal driver BLE quando si verificano eventi rilevanti
    ESP_ERROR_CHECK(esp_ble_gap_register_callback(esp_gap_cb));
    ESP_ERROR_CHECK(esp_ble_gatts_register_callback(esp_gatts_cb));

    ESP_ERROR_CHECK(esp_ble_gap_set_device_name("IOT_SENSOR")); // Imposta il nome del dispositivo BLE che sarà visibile durante l'advertising
    ESP_ERROR_CHECK(esp_ble_tx_power_set(ESP_BLE_PWR_TYPE_ADV, ESP_PWR_LVL_N12)); // Imposta il livello di potenza di trasmissione BLE per l'advertising
    ESP_ERROR_CHECK(esp_ble_gatts_app_register(PROFILE_ID)); // Registra l'applicazione GATTs con l'ID del profilo.
    //Questo innescherà l'evento ESP_GATTS_REG_EVT nella callback GATTs
}
```

Fig. 15: Codice di inizializzazione dello stack Bluetooth (Server)

Il GAP (Generic Access Profile) gestisce il modo in cui il server si presenta e comunica la propria presenza agli altri dispositivi. Il dispositivo si pubblicizza con parametri definiti in **ble_adv_params** e dati in **adv_data**.

```
// Parametri di advertising BLE. Controllano come il dispositivo si pubblicizza agli altri dispositivi BLE
static esp_ble_adv_params_t ble_adv_params = {
    .adv_int_min = 0x20, // Intervallo minimo di advertising (multiplo di 0.625ms, qui 32 * 0.625ms = 20ms)
    .adv_int_max = 0x40, // Intervallo massimo di advertising (multiplo di 0.625ms, qui 64 * 0.625ms = 40ms)
    .adv_type = ADV_TYPE_IND, // Tipo di advertising: connettabile e scansionabile
    .own_addr_type = BLE_ADDR_TYPE_PUBLIC, // Tipo di indirizzo BLE utilizzato (pubblico)
    .channel_map = ADV_CHNL_ALL, // Mappa dei canali di advertising (tutti i 3 canali primari)
    .adv_filter_policy = ADV_FILTER_ALLOW_SCAN_ANY_CON_ANY, // Policy di filtro per le scansioni e le connessioni (consente a tutti)
};

// Dati di advertising BLE. Definiscono quali informazioni saranno incluse nei pacchetti di advertising
static esp_ble_adv_data_t adv_data = {
    .include_name = true, // Include il nome del dispositivo nei dati di advertising
    .include_txpower = true, // Include il livello di potenza di trasmissione nei dati di advertising
    .flag = ESP_BLE_ADV_FLAG_LIMIT_DISC | ESP_BLE_ADV_FLAG_BREDR_NOT_SPT, // Flag specifici: limitatamente scansionabile, BR/EDR non supportato (solo BLE)
};
```

Fig. 16: Parametri e dati di advertising

Sono stati posti come minimo e massimo per l'intervallo di advertising rispettivamente 20 ms e 40 ms. L'intervallo di advertising effettivo sarà scelto casualmente tra il minimo e il massimo per prevenire collisioni con altri dispositivi.

esp_gap_cb è la callback che riceve gli eventi GAP come:

- **ESP_GAP_BLE_ADV_DATA_SET_COMPLETE_EVT:** quando i dati di advertising sono impostati.
- **ESP_GAP_BLE_ADV_START_COMPLETE_EVT:** quando l'advertising parte.
- **ESP_GAP_BLE_ADV_STOP_COMPLETE_EVT:** quando l'advertising si ferma.

Si avvia e ferma l'advertising per permettere o sospendere la connessione da parte di un client.

Il GATT, invece, definisce come i dati (attributi) sono organizzati, letti e scritti sul dispositivo. Nel nostro codice abbiamo creato un servizio GATT con UUID 0x00FF. Abbiamo poi definito tre caratteristiche:

- **Temperatura** (CHAR_UUID_TEMP)
- **Umidità** (CHAR_UUID_HUMI)
- **Luminosità** (CHAR_UUID_LUX)

Per ogni caratteristica, impostiamo permessi (solo lettura) e valori iniziali.

La callback **esp_gatts_A_cb** gestisce eventi GATT come:

- Registrazione del profilo,
- Creazione e avvio del servizio,
- Aggiunta delle caratteristiche,
- Gestione di lettura di caratteristiche (ESP_GATTS_READ_EVT),
- Connessione e disconnessione client.

Quando un client legge una caratteristica, il codice legge il valore dal sensore, lo converte in un formato adatto e risponde al client.

Il dato letto dal sensore viene opportunamente convertito prima di essere inviato al client come riportato di seguito:

```

// Converte il valore float del sensore in un int16_t moltiplicandolo per 100 e lo divide in due byte (LSB e MSB) per l'invio via BLE.
uint16_t val = (uint16_t)(sensor_value * 100);
data[0] = val & 0xFF; // Byte meno significativo
data[1] = (val >> 8) & 0xFF; // Byte più significativo

// Prepara la risposta GATT da inviare al client
esp_gatt_rsp_t rsp;
memset(&rsp, 0, sizeof(rsp)); // Inizializza la struttura a zero
rsp.attr_value.handle = handle; // Handle della caratteristica che è stata letta
rsp.attr_value.len = 2; // Lunghezza della risposta (2 byte)
rsp.attr_value.value[0] = data[0]; // LSB del valore
rsp.attr_value.value[1] = data[1]; // MSB del valore

// Invia la risposta GATT al client (non avendo messo la risposta automatica questa va inviata manualmente)
ESP_ERROR_CHECK(esp_ble_gatts_send_response(
    gatts_if, // Interfaccia GATT
    param->read.conn_id, // ID della connessione
    param->read.trans_id, // ID della transazione
    ESP_GATT_OK, // Stato della risposta (OK)
    &rsp)); // Puntatore alla struttura della risposta

```

Fig. 17: Codice di conversione del dato letto

Il valore letto da un sensore viene prima moltiplicato per 100 (così da evitare l'uso di virgola). Il risultato della moltiplicazione viene poi convertito tramite un cast a uint16_t. Questo significa che il valore sarà un intero senza segno a 16 bit, con un intervallo da 0 a 65535.

Dopodichè vengono preparati i due byte data[0] e data[1] che verranno inviati.

Viene poi dichiarata una variabile rsp di tipo esp_gatt_rsp_t. Questa è una struttura definita dall'ESP-IDF che incapsula tutti i dettagli necessari per costruire una risposta a una richiesta GATT da parte del client.

I due byte (data[0] e data[1]) che rappresentano il valore del sensore vengono copiati nei campi value[0] e value[1] dell'array rsp.attr_value.value. Questo array è dove i dati effettivi della caratteristica vengono memorizzati per essere inviati.

A questo punto abbiamo l'invio della risposta GATT al client tramite la funzione esp_ble_gatt_send_response().

2.3 Client ESP32

All'avvio del programma, inizia la fase di scanning. Fino a quando la connessione non viene stabilita, sul display verrà mostrato il seguente messaggio:

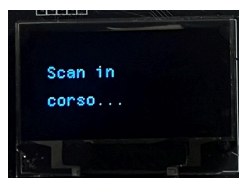


Fig. 18: Schermata di attesa di connessione (Client)

Una volta stabilita la connessione, il client richiede le misure di temperatura, umidità, e pressione. Queste vengono ricevute ogni 10 secondi, e vengono rappresentate su display nella seguente maniera:

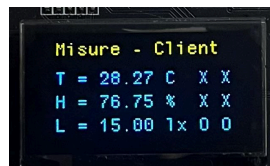


Fig. 19: Schermata di presentazione a connessione stabilita (Client)

Le due colonne rappresentano rispettivamente: la colonna di sinistra il caso dei libri antichi, e la colonna di destra quello dei libri moderni. Se il valore rilevato rientra nella soglia prevista, viene indicato con una 'O', altrimenti, con una 'X'.

Come per il server, l'inizializzazione dello stack Bluetooth e la configurazione delle funzioni di callback del driver si trovano nel main:

```
void app_main(void)
{
    ESP_ERROR_CHECK(nvs_flash_init());
    ESP_ERROR_CHECK(i2c_master_init());
    ESP_ERROR_CHECK(esp_bt_controller_mem_release(ESP_BT_MODE_CLASSIC_BT));

    //Inizializzazione controller Bluetooth (HCI, LL and PHY)
    esp_bt_controller_config_t bt_cfg = BT_CONTROLLER_INIT_CONFIG_DEFAULT();
    ESP_ERROR_CHECK(esp_bt_controller_init(&bt_cfg));
    ESP_ERROR_CHECK(esp_bt_controller_enable(ESP_BT_MODE_BLE));

    //Inizializzazione Stack Software (Creazione thread gestore del bluetooth)
    ESP_ERROR_CHECK(esp_bluedroid_init());
    ESP_ERROR_CHECK(esp_bluedroid_enable());

    printf("Stack Initialized\n\n");

    //Configurazione della funzione di callback del driver
    ESP_ERROR_CHECK(esp_ble_gap_register_callback(esp_gap_cb));
    ESP_ERROR_CHECK(esp_ble_gattc_register_callback(esp_gattc_cb));

    ESP_ERROR_CHECK(esp_ble_gattc_app_register(PROFILE_A_APP_ID));
}
```

Fig. 20: Codice di inizializzazione dello stack Bluetooth (Client)

Una volta stabilita la connessione con il server, ad un certo punto, viene generato l'evento di lettura delle caratteristiche di temperatura, umidità e pressione. Per quanto riguarda la temperatura:

```
case ESP_GATTC_SEARCH_CMPL_EVT:
{
    printf("ESP_GATTC_SEARCH_CMPL_EVT\n");
    if(get_server == true) { // Se il servizio è stato trovato con successo.
        uint16_t count = 0;
        esp_gattc_char_elem_t *char_elem_result = NULL;

        ESP_ERROR_CHECK(esp_ble_gattc_get_attr_count(gattc_if, param->search_cmpl.conn_id, ESP_GATT_DB_CHARACTERISTIC,
            gl_profile_tab[PROFILE_A_APP_ID].service_start_handle,
            gl_profile_tab[PROFILE_A_APP_ID].service_end_handle, IGNORE_HANDLE, &count));

        if (count > 0) {
            char_elem_result = (esp_gattc_char_elem_t *) malloc(sizeof(esp_gattc_char_elem_t) * count);
            if (char_elem_result) {
                // Cerca la caratteristica specifica della temperatura tramite UUID.
                ESP_ERROR_CHECK(esp_ble_gattc_get_char_by_uuid(gattc_if, param->search_cmpl.conn_id, gl_profile_tab[PROFILE_A_APP_ID].service_start_handle,
                    gl_profile_tab[PROFILE_A_APP_ID].service_end_handle, remote_char_uuid_temp, char_elem_result,
                    &count));

                if (count > 0) {
                    // Salva l'handle della caratteristica temperatura.
                    gl_profile_tab[PROFILE_A_APP_ID].char_handle_temp = char_elem_result[0].char_handle;
                    printf("Temperatura Handle: %d\n", gl_profile_tab[PROFILE_A_APP_ID].char_handle_temp);
                }
            }
        }
    }
}
```

Fig. 21: Codice di lettura dei dati inviati (temperatura)

Viene allocata una quantità di memoria dinamica adeguata. Dopodiché, a partire dall'UUID, viene trovata la caratteristica di interesse, e vengono salvate le relative informazioni nello spazio di memoria precedentemente allocato. Ciò viene eseguito per tutte e tre le caratteristiche. A questo punto, possono essere salvati gli handle e si possono fare le letture delle tre caratteristiche.

Per ricostruire e convertire il valore trasmesso dal server si procede come segue:

```
// Il server invia un uint16_t (due byte) rappresentante il valore del sensore moltiplicato per 100.  
// Ricostruisce il valore uint16_t dal LSB e MSB.  
uint16_t raw_value = (param->read.value[1] << 8) | param->read.value[0]; // Ricostruisce il valore grezzo (LSB, poi MSB).  
// Converte il valore grezzo in un float dividendo per 100.  
float sensor_value = (float)raw_value / 100.0f; // Converte in float e divide per 100.
```

Fig. 22: Codice di ricostruzione dei dati effettivi

Il primo passo consiste nel ricostruire il valore a 16 bit combinando i due byte ricevuti. Una volta ottenuto il valore grezzo, esso viene convertito in virgola mobile (float) e diviso per 100. Questa divisione serve a riportare il numero al suo valore reale, compensando la moltiplicazione effettuata dal server al momento dell'invio.

Infine per rappresentare i valori sul display, viene definita una funzione chiamata **show_value**. Questa si occupa di fornire la schermata mostrata in figura precedente.

2.4 Client smartphone

Per il client smartphone abbiamo implementato un'applicazione per iPhone tramite il programma Xcode. In particolare, il codice definisce una classe ViewController che gestisce l'interazione con il server.

Il programma utilizza i framework **UIKit** per l'interfaccia grafica e **CoreBluetooth** per la comunicazione BLE. Il ViewController eredita da UIViewController e implementa i protocolli delegati CBCentralManagerDelegate e CBPeripheralDelegate per gestire eventi legati al Bluetooth, come la scansione, la connessione e la lettura di dati dal dispositivo periferico.

Il flusso principale è il seguente:

1. All'avvio, si inizializza il manager Bluetooth (CBCentralManager) e si parte con la scansione dei dispositivi BLE.
2. Quando viene scoperto un dispositivo con nome "IOT_SENSOR", la scansione si ferma e si tenta la connessione.
3. Una volta connessi, si cercano servizi e caratteristiche specifiche identificate da UUID custom (servizio 00FF, caratteristiche FF01, FF02, FF03 per temperatura, umidità e luminosità).
4. Il dispositivo legge periodicamente il valore RSSI per aggiornare la qualità del segnale.

- Quando si ricevono aggiornamenti dei valori dai sensori, questi vengono convertiti, valutati rispetto a soglie configurabili tramite il controllo segmentato, e mostrati nella UI con un testo colorato che indica se il valore è “OK” o fuori range.
- È possibile leggere manualmente i dati premendo un pulsante, o connettersi/disconnettersi tramite un altro pulsante.
- La valutazione dei parametri confronta i valori con soglie predefinite, evidenziando eventuali anomalie con un colore rosso e un'icona di avviso.

2.4.1 Interfaccia applicazione

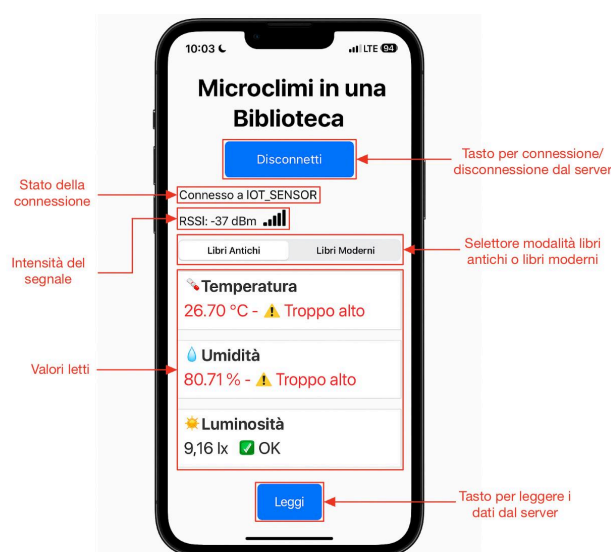


Fig. 23: Interfaccia applicazione

Nella parte superiore dell'interfaccia troviamo il **tasto per connettere o disconnettere il dispositivo dal server**, identificabile tramite il pulsante blu “Disconnetti”. Una volta stabilita la connessione, l'app mostra lo stato con la scritta “**Connesso a IOT_SENSOR**”, confermando all'utente che la comunicazione con il sensore IoT è attiva.

Subito sotto lo stato di connessione viene visualizzato il valore **RSSI (Received Signal Strength Indicator)**, espresso in dBm. Questo indicatore fornisce una misura quantitativa della **potenza del segnale radio ricevuto** dal dispositivo. A fianco, è presente un'icona che rappresenta graficamente la qualità del segnale, suddivisa in sei livelli, facilitando una valutazione immediata anche a livello visivo.

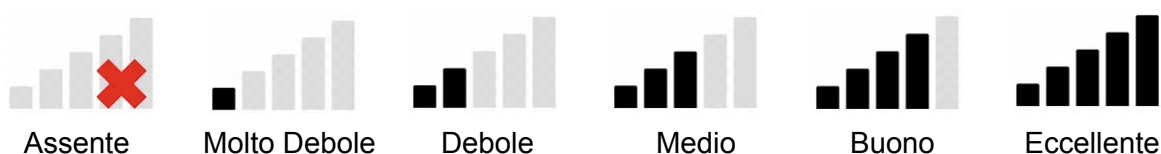


Fig. 24: Indici di qualità del segnale

Le soglie sono state scelte come segue:

- **Assente:** connessione non stabilita o RSSI < - 90 dBm
- **Molto Debole:** - 80 dBm < RSSI < - 90 dBm
- **Debole:** - 70 dBm < RSSI < - 80 dBm
- **Medio:** - 60 dBm < RSSI < - 70 dBm
- **Buono:** - 50 dBm < RSSI < - 60 dBm
- **Eccellente:** - 40 dBm < RSSI < - 50 dBm

L'interfaccia include un **selettore a due opzioni** per scegliere se monitorare le condizioni ideali per **Libri Antichi** oppure **Libri Moderni**.

L'area centrale dell'interfaccia riporta i **valori letti dal sensore**, suddivisi in tre categorie principali, una per ogni valore letto.

Infine, in basso è presente il **pulsante "Leggi"**, che permette all'utente di effettuare una **lettura manuale dei dati**.

3. Test e risultati

Sono stati condotti alcuni test per valutare l’RSSI in funzione della distanza e della potenza trasmessa. La potenza trasmessa dal server è modificabile direttamente dal codice tramite la seguente riga:

```
ESP_ERROR_CHECK(esp_ble_tx_power_set(ESP_BLE_PWR_TYPE_ADV, ESP_PWR_LVL_N12));
```

Fig. 25: Selezione del valore della potenza di trasmissione

Per il funzionamento normale è stata settata a -12 dBm, successivamente modificata per test ulteriori.

Inoltre sono stati condotti test sul consumo di potenza del server durante le attività di advertising e di comunicazione.

3.1 RSSI in funzione della distanza e della Ptx

Questo test serve a valutare come varia il valore dell’RSSI al cambiare della distanza e della potenza di trasmissione del server. In particolare, sono stati effettuati due tipi di test:

1. **In condizioni di Line of Sight (LoS):** sono stati utilizzati 4 diversi livelli di potenza e 4 distanze differenti.
2. **In condizioni di Non Line of Sight (NLoS):** con la potenza di trasmissione fissata a -12 dBm, è stato inserito un ostacolo (un muro) tra server e client e ripetute le misurazioni alle stesse distanze.

Di seguito i risultati ottenuti.

LoS

txPower [dBm]	RSSI [dBm] @ 1m	RSSI [dBm] @ 3m	RSSI [dBm] @ 6m	RSSI [dBm] @ 9m
-12	-63	-65	-75	-79
-6	-60	-64	-68	-71
0	-59	-62	-65	-69
6	-57	-60	-62	-67

Fig. 26: RSSI in condizioni di LoS

NLoS

txPower [dBm]	RSSI [dBm] @ 1m	RSSI [dBm] @ 3m	RSSI [dBm] @ 6m	RSSI [dBm] @ 9m
-12	-66	-74	-77	-81

Fig. 27: RSSI in condizioni di NLoS

Come possiamo osservare, L'RSSI diminuisce all'aumentare della distanza, come atteso. Questo comportamento è coerente con il modello di attenuazione del segnale in spazio libero. A parità di distanza, l'aumento della potenza di trasmissione comporta un miglioramento dell'RSSI.

Inserendo l'ostacolo L'RSSI è sistematicamente più basso rispetto ai corrispettivi valori LoS.

3.2 Potenza dissipata

L'obiettivo di questa analisi è valutare il consumo energetico del modulo ESP32 nelle due fasi operative. Viene inoltre effettuata una stima della durata della batteria in base alla potenza media dissipata.

I test sono stati effettuati esclusivamente sul lato server. Il client, infatti, viene considerato alimentato da una fonte energetica stabile, come ad esempio la rete elettrica, e quindi non soggetto a vincoli di autonomia. Il server, invece, potrebbe ragionevolmente essere alimentato da una batteria.

I test sono stati svolti nei seguenti scenari:

1. **Advertising BLE:** la scheda fa continuamente advertising.
2. **Connessione attiva BLE:** la scheda è connessa a un client e scambia dati.

Di seguito una tabella che riassume i risultati ottenuti:

Stato	Potenza [mW]	P _{avg} [mW]
Advertising	161 - 345	253
Connessione	161 - 485	323

Fig. 28: Potenza dissipata (Server)

Si osserva come si abbia un range di potenza dissipata durante i vari istanti del test.

Per stimare la durata del sistema ESP32 alimentato a batteria, si ipotizza l'utilizzo di una batteria commerciale di tipo coin cell, ad esempio una CR2032, comunemente utilizzata nei

dispositivi a basso consumo. Questa batteria fornisce una tensione nominale di circa 3,0 V e ha una capacità tipica di 220 mAh, pari a 660 mWh di energia disponibile.

Per stimare la durata della batteria, si prende in considerazione esclusivamente il contributo legato alla connessione. Infatti, la fase di advertising dura pochi secondi, mentre per il resto del tempo si ha la comunicazione vera e propria tra client e server.

Una volta stimata la potenza media, la durata stimata della batteria si ottiene tramite:

$$t_{lifetime} = \frac{E_{battery}}{P_{avg}} \quad (1)$$

Dove $E_{battery}$ è l'energia disponibile dalla batteria, calcolata come:

$$E_{battery} = V \cdot Q = 3 V \cdot 0.220 Ah = 0.66 Wh = 660 mWh \quad (2)$$

Quindi nel nostro caso:

$$t_{lifetime} = \frac{660 mWh}{323 mW} \simeq 2h \quad (3)$$

La valutazione della potenza consumata è stata effettuata mantenendo il display acceso. Questa scelta ha senso solo a livello di progetto, sia per motivi estetici sia per facilitare eventuali operazioni di debug. Tuttavia, in un'ottica di ulteriore risparmio energetico, è ovvio che il display possa essere spento durante il normale funzionamento.

Un modo efficace per aumentare l'efficienza energetica consiste nell'implementare una modalità di *sleep* per il nodo periferico nei periodi in cui non è necessario acquisire o trasmettere dati al nodo centrale.

3.2.1 Impatto del data rate sul consumo di potenza

Nel nostro sistema BLE, durante la connessione, il server invia dati al client su richiesta. Ogni richiesta comporta la trasmissione di 6 byte totali, suddivisi in 3 caratteristiche da 2 byte ciascuna.

Il tempo necessario per trasmettere i 6 byte dipende dal data rate BLE. Considerando un data rate di 1 Mbps:

- 1 Mbps = 1 milione di bit per secondo
- 1 (Preamble) + 4 (Access Address) + 2 (Header) + 2 (Payload) + 3 (CRC) = **12 byte**
- $12 \cdot 3 = 36 \text{ byte} \rightarrow 288 \text{ bit}$

- Tempo di trasmissione = $288 \text{ bit} \div 1,000,000 \text{ bit/s} = 288 \text{ } \mu\text{s}$ (microsecondi)

In pratica, la trasmissione dura meno di circa 0.3 ms.

Nel progetto abbiamo considerato il caso peggiore, ovvero l'invio di due byte per ciascun valore di ogni caratteristica. Per ridurre il data rate, e quindi il consumo di potenza durante la trasmissione, è possibile ottimizzare il sistema diminuendo il numero di byte inviati, ad esempio riducendo la risoluzione dei valori.

4. Antenna PIFA

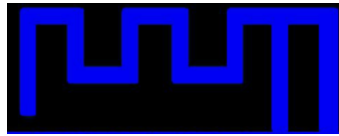


Fig. 29: Antenna PIFA

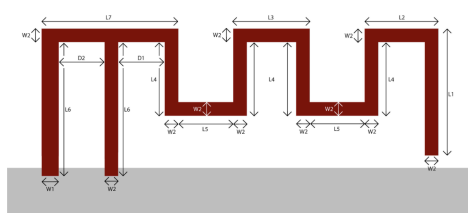
La **PIFA** è una variante dell'antenna ad "F" rovesciata. Su PCB, appare come una traccia metallica in rame sagomata in una forma a "meandro".

È composta da:

- **Elemento radiante:** la striscia di rame sul PCB.
- **Piano di massa.**
- **Corto circuito:** connette l'elemento radiante al piano di massa.
- **Alimentazione:** punto in cui l'antenna è collegata al circuito RF del modulo ESP32.

4.1 Simulazione con HFSS

La prima operazione effettuata è stata la progettazione del piano di massa, in rame e del substrato, in FR4, utilizzando spessori standard ovvero 70 μ m per il piano di massa e 1.6 mm per il substrato. Successivamente, abbiamo disegnato l'antenna basandoci sulle misure trovate in rete di seguito riportate.



Reference	mm
L1	4.67 mm
L2	2.7 mm
L3	2.8 mm
L4	2.7 mm
L5	2 mm
L6	4.91 mm
L7	5 mm
W1	0.63 mm
W2	0.5 mm
D1	1.7 mm
D2	1.67 mm

Fig. 30: Misure caratteristiche dell'antenna

Alla lunghezza della "gamba" finale L1 è stato assegnato il parametro 'L' in modo da effettuare degli sweep successivi.

Il risultato finale è il seguente:

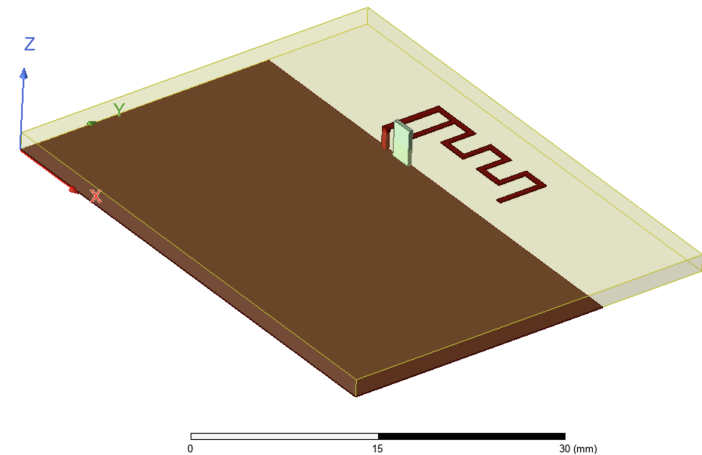


Fig. 31: Disegno dell'antenna

In particolare, uno dei terminali dell'antenna è stato collegato al piano di massa, mentre l'altro è stato connesso a una porta wave port, che rappresenta il punto in cui arriva il segnale da trasmettere.

Abbiamo quindi simulato il coefficiente di riflessione S_{11} dell'antenna, ottenendo una frequenza di risonanza pari a 2.3 GHz, leggermente inferiore rispetto al valore atteso di 2.4 GHz.

Per correggere questa discrepanza, è stato eseguito uno sweep della lunghezza L. Il primo sweep è stato effettuato con valori di L compresi tra 1 mm e 8 mm, con uno step di 1 mm. Il risultato ottenuto è il seguente:

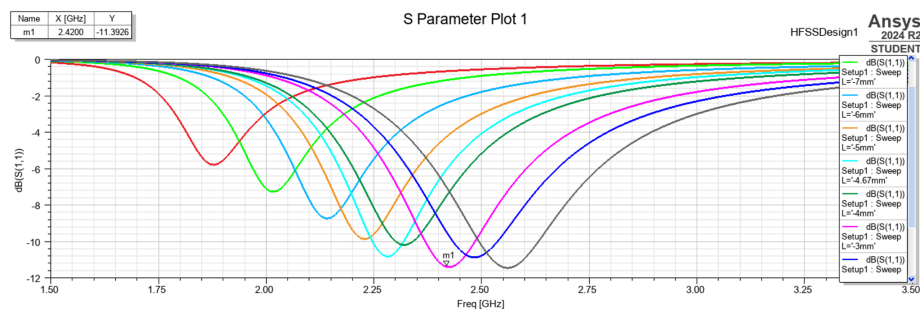


Fig. 32: Andamento in frequenza del coefficiente di riflessione S_{11} per diverse lunghezze (1)

Si osserva che il punto ottimale si trova intorno a 3 mm. A questo punto, abbiamo effettuato uno sweep più fine, variando L da 2.5 mm a 3.5 mm con incrementi di 0.2 mm, per individuare con maggiore precisione il valore ottimale. I risultati ottenuti sono i seguenti:

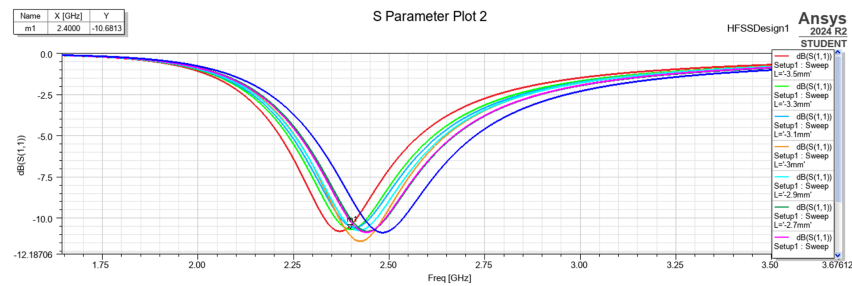


Fig. 33: Andamento in frequenza del coefficiente di riflessione S_{11} per diverse lunghezze (2)

Dalle simulazioni si evince che il valore ottimale di L è 3.3 mm, in corrispondenza del quale si ottiene la risonanza esattamente a 2.4 GHz.

Con $L = 3.3$ mm abbiamo infine determinato il minimo di S_{11} e la banda passante dell'antenna, fissando un marker a -10 dB. Abbiamo così ottenuto un minimo pari a -10.68 dB e una banda compresa tra 2.3598 GHz e 2.4425 GHz.

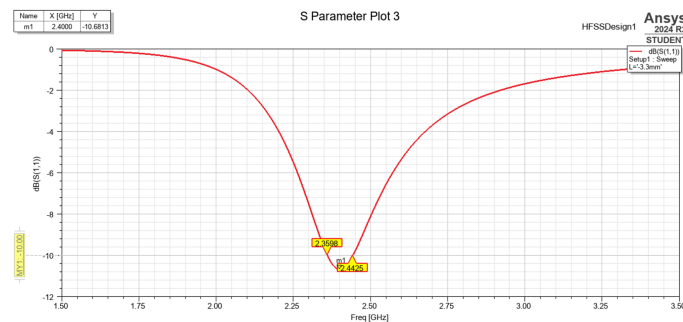


Fig. 34: Determinazione della banda passante dell'antenna

Finora abbiamo ottimizzato la lunghezza L concentrandoci esclusivamente sul parametro S_{11} , senza considerare l'adattamento in ingresso. Per ottenere risultati migliori, possiamo cercare di adattare l'impedenza dell'antenna a quella della wave port.

Abbiamo quindi valutato l'impedenza di ingresso dell'antenna alla frequenza di **2.4 GHz**, ottenendo i seguenti risultati:

Parte Reale: 94.72 Ω

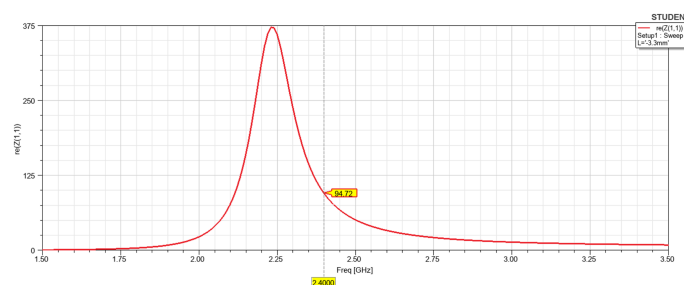


Fig. 35: Andamento in frequenza dell'impedenza di ingresso (parte reale) dell'antenna

Parte Immaginaria: -60.50Ω

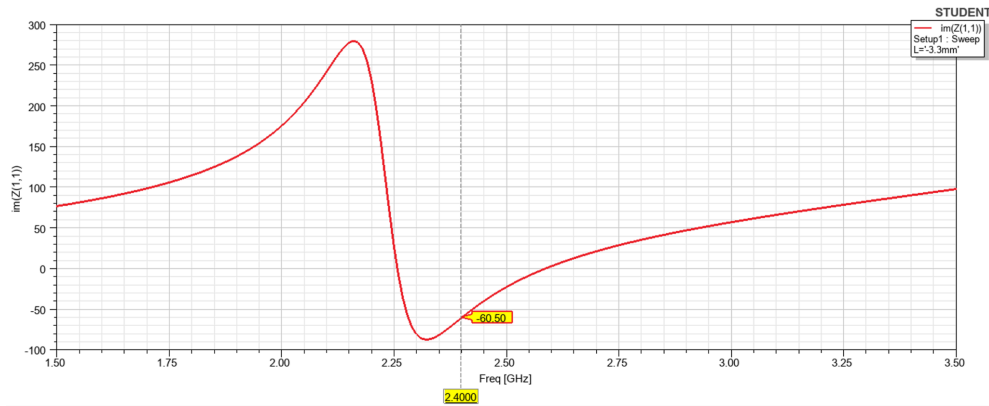


Fig. 36: Andamento in frequenza dell'impedenza di ingresso (parte immaginaria) dell'antenna

Da ciò, risulta: $Z_{11} = 94.72 - 60.50j$.

Impostando l'impedenza della sorgente uguale al complesso coniugato dell'impedenza dell'antenna otteniamo il seguente risultato:

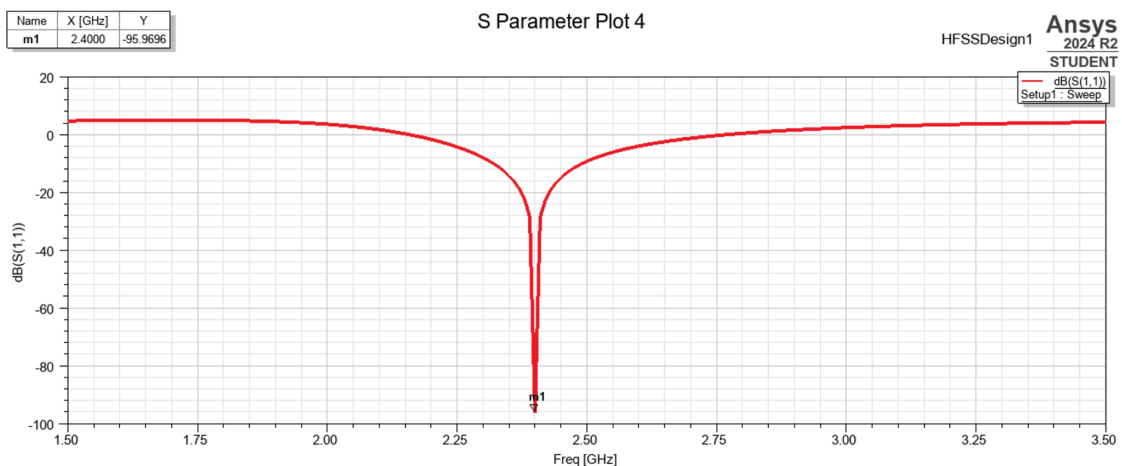


Fig. 37: Andamento in frequenza del coefficiente di riflessione S_{11} in condizioni di adattamento c.c

Notiamo un netto miglioramento del parametro S_{11} rispetto al caso precedente.

In alternativa, sarebbe stato possibile modificare vari parametri, come la lunghezza della gamba in corrispondenza della quale si inietta il segnale RF, in modo da ottenere un'impedenza dell'antenna pari a 50Ω . Oppure, si potrebbe semplicemente inserire una rete di adattamento. Questo potrebbe non essere necessario se, ad esempio, il circuito che sfrutta l'antenna viene progettato dal progettista dell'antenna stessa.

Di seguito è riportata la rappresentazione 3D e 2D del diagramma di irradiazione:

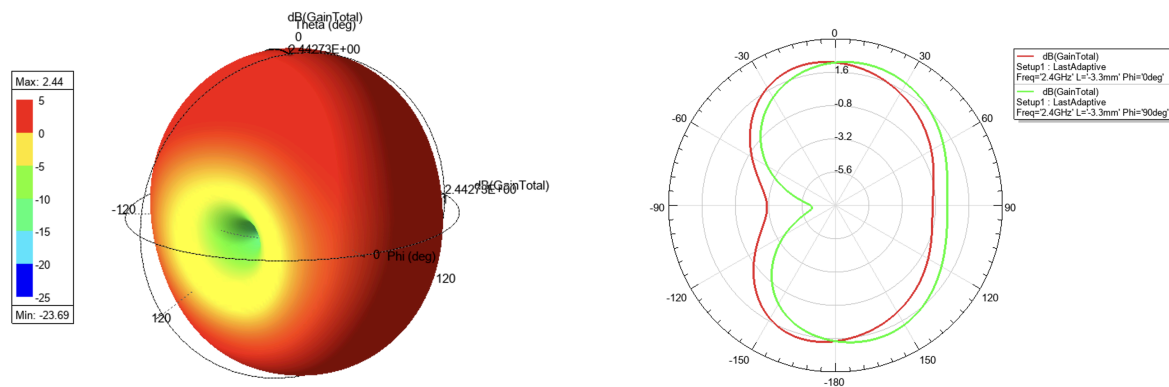


Fig. 38: Diagrammi di irradiazione

A sinistra è mostrata la rappresentazione tridimensionale del guadagno totale (in dB), da cui si osserva un massimo di circa 2.44 dB. La distribuzione è tipica di una PIFA, con un pattern direzionale e una buona copertura nella direzione desiderata.

A destra sono riportati i diagrammi di irradiazione in coordinate polari nei piani $\phi=0$ (in rosso) e $\phi=90$ (in verde).

5. Altre soluzioni sul mercato

Sul mercato sono presenti soluzioni per il monitoraggio di ambienti come le biblioteche. Un esempio è **Testo Saveris 2**, che si tratta di un sistema di **data logger Wi-Fi** per il monitoraggio di temperatura e umidità.

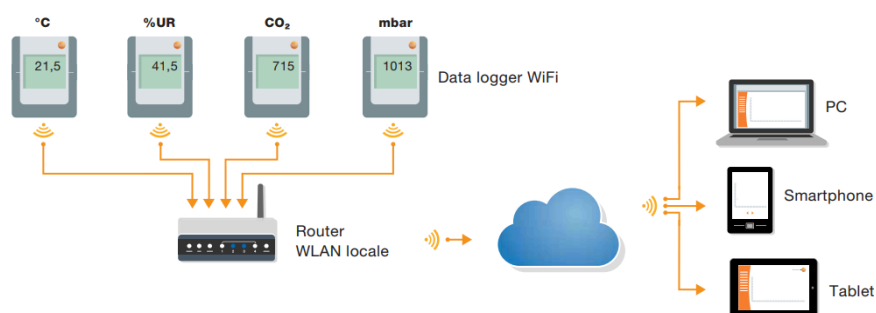


Fig. 39: Generico sistema di monitoraggio ambientale

Vengono utilizzati dei piccoli dispositivi che misurano continuamente temperatura, umidità, e/o altri parametri. Questi dispositivi inviano automaticamente e a intervalli configurabili (ad esempio, ogni 5, 10, 15 minuti) i dati misurati a un **cloud Testo dedicato**. Ciò consente di poter accedere ai dati da qualunque luogo, a patto di avere accesso ad internet.

Con questo sistema è possibile effettuare misure di temperatura e umidità con tolleranze di ± 0.5 °C e $\pm 2\%$. Viene prevista una durata della batteria di 12 mesi, a +25 °C, con ciclo di misura di 15 min.

6. Conclusioni

Il progetto ci ha permesso di analizzare nel dettaglio il funzionamento del BLE, con particolare attenzione al consumo energetico. Attraverso lo studio della potenza assorbita e il confronto con la durata della batteria, abbiamo compreso quanto sia importante adottare tecniche *low power* e *low energy* in applicazioni di questo tipo, dove il risparmio energetico rappresenta un fattore determinante.

7. Bibliografia

- **Espressif (ESP32):** <https://www.espressif.com/en/products/socs/esp32>
- **HTS221:** <https://www.st.com/resource/en/datasheet/hts221.pdf>
- **BH1750FVI:** <https://www.mouser.com/datasheet/2/348/bh1750fvi-e-186247.pdf?srlti d=AfmBOopCUhfmkBHbkjVxZYpOLc8UI8e3GIMVhfBNxWPChW9nPpujlnpb>
- **PIFA:** https://github.com/prasad-dot-ws/ESP32_MIFA_PCB_ANTENNA
- **Testo Saveris 2:** <https://www.testo.com/it-IT/applicazioni/museum-saveris-2>